

Two critical corrections

The MFCentral CAS API is dead. Per your own uploaded doc, SEBI/AMFI shut it down in September 2025, stranding 100+ fintechs that relied on it — including the exact PAN + OTP sync flow MProfit and Investwell used. So "recreate what MProfit does" is not an option anymore; MProfit itself has had to migrate off it. Also worth saying plainly: attempting to hit MFCentral's now-private endpoints or automate its OTP flow without authorization would be unauthorized access, so that path is out both practically and legally.

The parse.bot API you flagged as "most important" does not return user portfolio data. It's a scraper of MFCentral's public pages — AMC lists, ~6,000 scheme details, NAVs, sector allocations. Useful as a fund-metadata/enrichment source, but it cannot tell you what a user holds. It's a complement, not the core.

So what's actually the best path for a prototype? Ranked:

Path	User data?	Cost/effort	Verdict	CAS PDF parsing (open-source casparser Python lib)	✓	Full transaction history	Free, ~a day
Build this now	— user downloads their own CAS from CAMS/MFCentral, uploads it. Fully legitimate (their data, their consent), works today, and it's the same fallback the whole industry moved to	casparser.in hosted API	✓	₹999/mo, free 10 credits	Fallback/benchmark against your own parser	mfapi.in	✗ (NAV/scheme data only)
Free, no key	Use for enrichment — daily NAVs, historical NAV series for valuation charts	parse.bot MFCentral API	✗ (public scheme data)	Key needed	Optional enrichment (sector/holdings data)	smallcase Gateway	✓
Commercial partnership, KYB	Not viable for a test build	Account Aggregator (via TSP like Setu/Finvu)	✓	real-time	Regulated entity, 5–10 months, ₹5–25L	Your long-term differentiator, not the prototype	

This ranking is exactly the "CASParser now, AA later" strategy your doc recommends — and honestly it's the right one. And your "even better than MProfit" angle falls out naturally: MProfit's flow required OTP round-trips through a distributor partnership; a CAS-upload flow with a great verification UX + automatic NAV enrichment + XIRR/gains computed instantly is simpler for users and 100% yours.

I've written the full Claude Code prompt as a spec doc — architecture, stack, parsing pipeline, decimal-safe math rules, XIRR, capital-gains logic (equity/debt, grandfathering), and the minimal UI:
Created a file, read a file
Created a file, read a file
The prompt is ready to paste into Claude Code. The core architecture: CAS PDF upload → open-source casparser → mfapi.in NAV enrichment → Decimal-safe XIRR/FIFO gains → review-before-commit UI. Zero API keys, zero regulatory friction, works today.

What I need from you to make this real:

A test CAS PDF — this is the only genuine blocker. Request a Detailed CAS from camsonline.com against your own PAN (takes ~10 minutes to arrive by email). Without a real fixture file, Claude Code will be building blind against the parser.

Nothing else for v1 — mfapi.in is keyless. The casparser.in free tier and parse.bot key are optional nice-to-haves for benchmarking/enrichment later.

One strategic note worth sitting with: the MF Central shutdown is actually good news for you. MProfit's OTP-sync advantage just got vaporized by regulators, and everyone is back to CAS parsing + AA. That means the near-term playing field is level (parsing UX quality wins), and the medium-term moat is AA integration — which has a 5–10 month regulated-entity runway, so whoever starts that clock first wins. If the startup is serious, the AA/TSP conversation (Setu, Finvu, Onemoney) should start in parallel with this prototype, not after it.

Also flagging for your case-study doc: the "MF Central CAS API integration" section we wrote earlier is now historical — MProfit's current import path post-Sept-2025 is worth re-verifying whenever you next look at them.

Claude Code Prompt — MF Portfolio Import Prototype (CAS-first)

Copy everything below the line into Claude Code as your kickoff prompt.

Context

I'm building a prototype of the mutual-fund import module for a wealth-management platform (an MProfit competitor). MF Central's CAS API was shut down by SEBI/AMFI in Sept 2025, so the industry-standard approach now is: **user uploads their own CAS (Consolidated Account Statement) PDF** → **we parse it** → **enrich with live NAV data** → **compute holdings, XIRR, and capital gains**. Build exactly that, end to end, with a minimal but clean UI.

Stack

- **Backend:** Python 3.11+, FastAPI, [casparser](#) (open-source, MIT — parses CAMS & KFinTech detailed CAS PDFs into structured JSON), [httpx](#) for NAV fetching, [python-dateutil](#).
- **Money math:** `decimal.Decimal` everywhere. NEVER float for amounts, units, NAVs, or gains. Units to 3 decimal places, amounts to 2, NAV to 4. This is non-negotiable.
- **DB:** SQLite via SQLAlchemy (prototype only; schema should be Postgres-portable — use Numeric column types, not Float).
- **Frontend:** Single-page React (Vite) or plain HTML+htmx — whichever is faster. Minimal: upload screen, processing state, portfolio dashboard. No auth for the prototype.
- **NAV enrichment:** mfapi.in (free, no key): `GET https://api.mfapi.in/mf/{scheme_code}` for full NAV history, `/latest` for current NAV. Cache responses locally (NAVs update once daily — do not hammer the API).

Core flow

1. **Upload:** User uploads a CAS PDF (CAMS "Detailed" CAS or MFCentral-generated detailed CAS) + the PDF password they set when requesting it. Show clear helper text explaining how to get one (camsonline.com → Statements → CAS, choose *Detailed*, period = since first investment).
2. **Parse:** Run `casparser` on the file. Handle both CAMS and KFintech layouts (the lib does this). Extract: investor info (name, email, PAN — mask PAN in UI as ABCDE****F), folios, schemes (with ISIN + AMFI scheme code where present), and every transaction (date, description, amount, units, NAV, type: PURCHASE / PURCHASE_SIP / REDEMPTION / SWITCH_IN / SWITCH_OUT / DIVIDEND_PAYOUT / DIVIDEND_REINVEST / SEGREGATION / STAMP_DUTY / STT / MISC).
3. **Normalize & store:** One canonical `transactions` table. Map casparser's transaction types into our enum. Store raw parser output too (JSON column) for debugging. Dedupe on (folio, scheme, date, amount, units) so re-uploading an overlapping CAS doesn't double-count — report "n new, m duplicates skipped" to the user.
4. **Enrich:** For each scheme, resolve the AMFI scheme code (casparser usually provides it; if missing, fuzzy-match scheme name against mfapi.in's scheme list `GET https://api.mfapi.in/mf` and flag low-confidence matches for user confirmation in the UI — never silently guess). Pull latest NAV + historical NAV series.
5. **Compute** (all Decimal):
 - Current holdings per scheme: net units × latest NAV.
 - Invested amount (net of redemptions, FIFO cost basis).
 - **XIRR** per scheme and portfolio-wide: implement Newton-Raphson with bisection fallback on the cashflow series (outflows = purchases negative, inflows = redemptions + current value as terminal inflow). Handle edge cases: single cashflow, all-negative, non-convergence → show "—" not a crash.
 - **Capital gains (India):** FIFO lot matching. Equity funds: STCG ≤ 12 months, LTCG > 12 months, apply the ₹1.25L LTCG exemption note (display, don't tax-file). Debt funds bought after Apr 1 2023: all gains taxed at slab (no indexation) — just classify and label correctly. Include grandfathering note for pre-Feb 1 2018 equity purchases (use Jan 31 2018 NAV as substituted cost where beneficial — fetch that NAV from the historical series). Keep this as a clearly-labeled *estimate*, with a disclaimer that it is not tax advice.
6. **Dashboard UI:**
 - Portfolio summary card: current value, invested, absolute gain, XIRR.
 - Holdings table: scheme, folio, units, avg cost, current NAV, value, gain, XIRR. Sortable.
 - Allocation donut: equity vs debt vs hybrid vs other (derive from scheme category via mfapi.in metadata).
 - Per-scheme drill-down: transaction list + NAV line chart (historical series) with buy/sell markers.

- Import history panel: file name, date, schemes found, txns added/skipped, and any low-confidence scheme matches awaiting confirmation.
7. **The "better than MProfit" touches** (cheap but differentiating):
- Confidence-scored import review screen: show what we parsed *before* committing — user confirms, we save. (MProfit imports silently and users discover errors later.)
 - Value-at-date: historical portfolio valuation line computed from NAV history × unit balance over time.
 - Clear empty/error states: wrong password, scanned/image-only PDF, summary-CAS-instead-of-detailed — each gets a specific, human error message telling the user exactly what to do.

Project structure

mf-import/

backend/

app/main.py # FastAPI app

app/parser.py # casparser wrapper + normalization

app/enrich.py # mfapi.in client with on-disk cache

app/calc.py # XIRR, FIFO gains, valuations (pure functions, Decimal)

app/models.py # SQLAlchemy models

app/schemas.py # Pydantic schemas

tests/ # pytest: XIRR known-answer tests, FIFO tests, parser fixture tests

frontend/ # minimal SPA

README.md # setup + how to get a CAS PDF

Rules

- Write `calc.py` as pure functions with pytest known-answer tests FIRST (e.g., XIRR of a simple SIP series verified against a spreadsheet value; FIFO gains on a hand-computed 4-transaction fixture). Then wire the rest.
- Uploaded PDFs: process in memory / temp dir, delete after parsing. Never log PAN, never store the PDF password.

- All API responses that include money: strings of Decimals, not JSON floats.
- If casparser fails on a file, fall back to reporting the failure cleanly with the parser's error — do NOT attempt to hand-roll PDF text extraction in v1.
- Commit in logical increments with clear messages.

Start by scaffolding the backend, writing calc.py + its tests, then the parser wrapper, then the API routes, then the frontend. Ask me before adding any dependency not listed above.

What I (the human) need to prepare

1. **A real test CAS:** request a *Detailed* CAS from camsonline.com (Statements → CAS → Detailed, period = all-to-date) to your email; you'll set a password — keep it. This is your primary test fixture. Get one KFintech-flavored CAS too if possible (via MFCentral's statement request) to test both layouts.
2. **No API keys needed** for the core build (mfapi.in is keyless). Optional: casparser.in free tier (10 credits/mo) if you want to benchmark your parse output against theirs; parse.bot key if you later want sector/holdings enrichment.
3. **Later (post-prototype):** shortlist an AA TSP (Setu / Finvu / Onemoney) and start the regulatory conversation early — FIU-relying-entity routes and timelines are the long pole for the real product.

name: MF CAS Import Prototype

overview: Greenfield build of `mf-import/` — a CAS PDF upload → parse → NAV enrich → compute → review → dashboard pipeline using FastAPI + casparser + mfapi.in on the backend, and Vite + React + Recharts on the frontend. Start with pure `calc.py` + pytest fixtures, then wire parser, API, and UI.

todos:

- id: scaffold

content: "Create mf-import/ monorepo: backend FastAPI skeleton, frontend

Vite+React+Recharts, requirements.txt, .gitignore"

status: completed

- id: calc-tests

content: Implement calc.py pure functions (XIRR, FIFO, value-at-date) + pytest

known-answer fixtures FIRST

status: completed

- id: models-parser

content: SQLAlchemy models (Numeric columns, dedupe) + parser.py casparser wrapper +
txn type mapping + error classification
status: completed
- id: enrich-cache
content: enrich.py mfapi.in httpx client, 24h disk cache, difflib fuzzy scheme matching with
confidence scores
status: completed
- id: api-routes
content: FastAPI two-phase import (parse/confirm), portfolio endpoints, Decimal-as-string
schemas, CORS
status: completed
- id: frontend-ui
content: Upload + Import Review + Dashboard (summary, sortable holdings, Recharts
donut/line, import history, error states)
status: in_progress
- id: readme
content: README with setup, CAS request instructions, test fixture guidance
status: pending
isProject: false

MF Portfolio Import Prototype (CAS-first)

Context

Workspace `[D:\WealthOS\CAS Parsers](D:\WealthOS\CAS Parsers)` is empty — full greenfield
scaffold per your spec under `mf-import/`. No auth; single implicit portfolio (SQLite prototype,
Postgres-portable schema).

****Approved extra frontend dep (per your choice):**** `recharts` (+ standard Vite/React toolchain).

****Implicit backend deps**** (standard FastAPI stack — confirm before adding anything else):
`uvicorn`, `sqlalchemy`, `python-multipart`, `pytest`, `pytest-asyncio`, `aiosqlite` (or sync SQLite
for simplicity).

****Fuzzy scheme matching:**** stdlib `difflib.SequenceMatcher` only — no `rapidfuzz`.

Architecture

```mermaid  
flowchart LR

```
subgraph frontend [Frontend_ViteReact]
 Upload[UploadScreen]
 Review[ImportReview]
 Dash[Dashboard]
end
```

```
subgraph backend [Backend_FastAPI]
 API[main.py_routes]
 Parser[parser.py]
 Enrich[enrich.py]
 Calc[calc.py]
 DB[(SQLite_SQLAlchemy)]
 Cache[(nav_cache_files)]
end
```

```
subgraph external [External]
 CASPDF[CAS_PDF]
 MFAPI[mfapi.in]
 CASParserLib[casparser]
end
```

```
Upload -->|PDF_plus_password| API
API --> Parser
Parser --> CASParserLib
Parser --> Enrich
Enrich --> MFAPI
Enrich --> Cache
API --> Calc
Calc --> API
Review -->|confirm| API
API --> DB
Dash --> API
CASPDF --> Upload
...
```

### ### Two-phase import (MProfit differentiator)

1. `POST /api/imports/parse` — upload PDF + password → parse in temp dir → enrich NAV metadata → return `preview` (no DB writes except optional session id in memory/temp).
2. User reviews schemes, confirms low-confidence AMFI matches.
3. `POST /api/imports/confirm` — dedupe + persist transactions → return `{added, skipped}`.

Password never stored; PDF deleted in `finally` block. PAN never logged; API masks as `ABCDE\*\*\*\*F`.

---

## Project layout

...

```
mf-import/
 backend/
 app/
 main.py # FastAPI app, CORS, routes
 parser.py # casparser wrapper + normalization
 enrich.py # mfapi.in client + disk cache
 calc.py # pure Decimal math
 models.py # SQLAlchemy ORM
 schemas.py # Pydantic DTOs (money as str)
 db.py # engine, session, init
 decimal_utils.py # quantize helpers (units 3, amt 2, nav 4)
 tests/
 test_calc_xirr.py
 test_calc_fifo.py
 test_calc_valuation.py
 test_parser_normalize.py # uses fixture JSON once CAS available
 requirements.txt
 pyproject.toml or pytest.ini
 frontend/
 src/
 App.tsx
 api/client.ts
 pages/Upload.tsx, Review.tsx, Dashboard.tsx
 components/...
 package.json
 README.md
```

...

---

## Phase 1 — Scaffold + `calc.py` + tests (FIRST)

Write `[backend/app/calc.py](mf-import/backend/app/calc.py)` as **pure functions** with typed inputs/outputs using `Decimal` and `date`. No DB, no HTTP.

| Function                                                                       | Responsibility                                                                                                                                                      |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -----                                                                          | -----                                                                                                                                                               |
| -----                                                                          | -----                                                                                                                                                               |
| <code>`quantize_units/amount/nav`</code>                                       | Enforce 3/2/4 decimal places via shared helpers                                                                                                                     |
| <code>`build_cashflows(transactions, terminal_value, as_of)`</code>            | Purchases negative, redemptions/dividends positive, terminal MV inflow                                                                                              |
| <code>`xirr(cashflows)`</code>                                                 | Newton-Raphson on NPV derivative; bisection on <code>`[-0.9999, 10]`</code> if NR fails; return <code>`None`</code> for single flow, all-same-sign, non-convergence |
| <code>`compute_holdings(transactions)`</code>                                  | Net units per scheme+folio                                                                                                                                          |
| <code>`fifo_cost_basis(transactions)`</code>                                   | Remaining lots + avg cost                                                                                                                                           |
| <code>`fifo_realized_gains(transactions, scheme_category, nav_history)`</code> | STCG/LTCG buckets; debt post-2023-04-01 slab label; equity grandfathering (Jan 31 2018 NAV substitute when beneficial)                                              |
| <code>`value_at_dates(unit_events, nav_series)`</code>                         | Daily/weekly portfolio MV timeline                                                                                                                                  |
| <code>`allocation_by_category(holdings, categories)`</code>                    | equity/debt/hybrid/other weights                                                                                                                                    |

### ### Known-answer pytest fixtures

**\*\*XIRR\*\*** — hand-verified SIP series (e.g. monthly ``-5000`` × 12 + terminal ``65000`` on day 365 → assert ~8–12% band against spreadsheet, tolerance ``0.0001``).

**\*\*FIFO\*\*** — 4-transaction fixture:

- Buy 100 @ 10 (2024-01-01)
- Buy 50 @ 12 (2024-06-01)
- Sell 80 @ 15 (2025-01-01) → matches first lot partially
- Assert STCG/LTCG split, realized gain = ``Decimal(...)`` computed by hand

**\*\*Edge cases:\*\*** ``xirr([])``, single cashflow, all-negative → ``None`` (UI shows ``"—``).

Commit: ``feat(calc): add Decimal XIRR, FIFO, and valuation pure functions with tests``

---

## Phase 2 — Data model + parser wrapper

### SQLAlchemy models (`[models.py](mf-import/backend/app/models.py)`)

Postgres-portable: `Numeric(precision, scale)` everywhere — **never** `Float`.

| Table          | Key fields                                                                                                         |  |
|----------------|--------------------------------------------------------------------------------------------------------------------|--|
| -----          |                                                                                                                    |  |
| -----          |                                                                                                                    |  |
| `imports`      | filename, imported_at, schemes_found, txns_added, txns_skipped, raw_json, status                                   |  |
| `schemes`      | name, isin, amfi_code (nullable until confirmed), category, match_confidence, match_status (`confirmed`/`pending`) |  |
| `folios`       | folio_number, amc, pan_masked                                                                                      |  |
|                |                                                                                                                    |  |
| `transactions` | scheme_id, folio_id, date, type (enum), description, amount, units, nav, dedupe_hash                               |  |
| `investor`     | name, email, pan_masked (single row for prototype)                                                                 |  |
|                |                                                                                                                    |  |

**\*\*Dedupe key:\*\*** SHA256 of `(folio, scheme\_name\_or\_id, date, amount, units)` — unique index on `dedupe\_hash`.

### Parser (`[parser.py](mf-import/backend/app/parser.py)`)

Wrap `casparser.read\_cas\_pdf(path, password)` → reject non-MF demat CAS early if `NSDLCASData`.

**\*\*Transaction type mapping\*\*** (casparser → canonical):

| casparser                                 | ours                          |  |
|-------------------------------------------|-------------------------------|--|
| -----                                     | -----                         |  |
| `PURCHASE`, `PURCHASE_SIP`                | same                          |  |
| `REDEMPTION`, `SWITCH_`*                  | same                          |  |
| `DIVIDEND_`, `SEGREGATION`                | same                          |  |
| `STT_TAX`                                 | `STT`                         |  |
| `STAMP_DUTY_TAX`                          | `STAMP_DUTY`                  |  |
| `TDS_TAX`, `GIFT_`, `REVERSAL`, `UNKNOWN` | `MISC` (preserve description) |  |

**\*\*Error classification\*\*** (human messages in  
`[schemas.py](mf-import/backend/app/schemas.py)`):

- Wrong password → "Incorrect PDF password. CAMS/KFintech CAS passwords are usually your PAN in uppercase."
- `cas\_type == SUMMARY` → "This is a Summary CAS. Request a **\*\*Detailed\*\*** CAS from camsonline.com → Statements → CAS."
- Parse exception mentioning encryption/image → "PDF appears scanned or unreadable. Download the original email PDF, not a photo/scan."
- Generic casparser error → pass through sanitized message

Store full parser output as JSON on import record for debugging.

Commit: `feat(parser): casparser wrapper, txn normalization, and SQLAlchemy schema`

---

### ## Phase 3 — NAV enrichment + cache

`[enrich.py](mf-import/backend/app/enrich.py)`:

- **\*\*Scheme list cache:\*\*** `GET https://api.mfapi.in/mf` → `backend/.cache/schemes.json` (TTL 24h)
- **\*\*NAV history cache:\*\*** `GET /mf/{code}` and `/latest` → `.cache/nav/{code}.json` (TTL until next calendar day IST)
- httpx async client with timeout + graceful degradation
- **\*\*Resolve AMFI code:\*\***
  1. Use casparser `amfi` if present → confidence `1.0`
  2. Else fuzzy-match normalized scheme name vs cached list → `SequenceMatcher.ratio()`
  3. `>= 0.92` → auto-suggest but still flag if `< 0.98` for review; `< 0.92` → `pending` — **\*\*never auto-save without confirm\*\***
- Parse `meta.scheme\_category` for allocation donut (map to equity/debt/hybrid/other via keyword rules)

Commit: `feat(enrich): mfapi.in client with disk cache and fuzzy scheme matching`

---

### ## Phase 4 — API routes

`[main.py](mf-import/backend/app/main.py)` endpoints:

| Method | Path                               | Purpose                                 |
|--------|------------------------------------|-----------------------------------------|
| POST   | `/api/imports/parse`               | Upload multipart; return preview DTO    |
| POST   | `/api/imports/confirm`             | Persist after review; return counts     |
| POST   | `/api/schemes/{id}/confirm-match`  | User picks correct AMFI code            |
| GET    | `/api/portfolio/summary`           | totals + XIRR                           |
| GET    | `/api/portfolio/holdings`          | sortable list                           |
| GET    | `/api/portfolio/allocation`        | category weights                        |
| GET    | `/api/schemes/{id}`                | drill-down: txns + NAV series + markers |
| GET    | `/api/portfolio/valuation-history` | value-at-date series                    |
| GET    | `/api/imports`                     | import history                          |

**Money serialization:** Pydantic `field_serializer` → all `Decimal` fields as strings in JSON responses.

**CORS:** allow Vite dev origin (`http://localhost:5173`).

Wire `calc.py` in service layer after fetch from DB; recompute on read for prototype (acceptable for SQLite scale).

Commit: `feat(api): two-phase import flow and portfolio endpoints`

---

## ## Phase 5 — Frontend (Vite + React + Recharts)

Minimal clean UI — no design system required; simple CSS modules or Tailwind only if you approve Tailwind (otherwise plain CSS).

### ### Screens

- Upload** (`[Upload.tsx](mf-import/frontend/src/pages/Upload.tsx)`)
  - PDF file input + password field
  - Helper text: camsonline.com → Statements → CAS → Detailed → all-to-date
  - Processing spinner → navigate to Review
- Import Review** (`[Review.tsx](mf-import/frontend/src/pages/Review.tsx)`) — *\*differentiator\**
  - Parsed investor (masked PAN), folio/scheme counts
  - Table of schemes with confidence badge; dropdown to fix low-confidence matches (calls mfapi search or cached list)
  - Confirm / Cancel buttons
- Dashboard** (`[Dashboard.tsx](mf-import/frontend/src/pages/Dashboard.tsx)`)

- Summary cards: current value, invested, gain, XIRR ("" when null)
  - Sortable holdings table (client-side sort)
  - Recharts `PieChart` allocation donut
  - Recharts `LineChart` value-at-date + per-scheme NAV chart with buy/sell scatter markers
  - Import history panel
  - Tax gains section with disclaimer banner (estimate only, ₹1.25L LTCG note, debt post-Apr-2023 label)
4. **Empty/error states** — dedicated components per error code from API

Commit: `feat(frontend): upload, review, and portfolio dashboard`

---

### ## Phase 6 — README + dev workflow

`[README.md](mf-import/README.md)`:

- Python 3.11+ venv setup, `pip install -r requirements.txt`
- `uvicorn app.main:app --reload` from `backend/`
- `npm run dev` from `frontend/`
- How to request Detailed CAS (CAMS + KFintech)
- Note: place test PDF locally (gitignored); never commit CAS files
- `.gitignore`: `\*.pdf`, `.cache/`, `\*.db`, `.env`

---

### ## Testing strategy

| Layer       | Tests                                                                                                       |  |
|-------------|-------------------------------------------------------------------------------------------------------------|--|
| -----       | -----                                                                                                       |  |
| `calc.py`   | Known-answer XIRR, FIFO, valuation (no external deps)                                                       |  |
|             |                                                                                                             |  |
| `parser.py` | Normalization unit tests with saved JSON fixture (after you provide CAS); optional live PDF test gitignored |  |
| API         | 1–2 integration tests with fixture JSON + in-memory SQLite                                                  |  |
|             |                                                                                                             |  |
| Frontend    | Manual for prototype; optional smoke test later                                                             |  |
|             |                                                                                                             |  |

**Your action item:** Request Detailed CAS from camsonline.com (and KFintech if possible). Place in `backend/tests/fixtures/` (gitignored) for parser integration tests.

---

### ## Commit sequence (logical increments)

1. `chore: scaffold mf-import backend and frontend`
2. `feat(calc): Decimal XIRR, FIFO, valuation with pytest fixtures`
3. `feat(db): SQLAlchemy models and dedupe strategy`
4. `feat(parser): casparser wrapper and error mapping`
5. `feat(enrich): mfapi.in cache and scheme resolution`
6. `feat(api): two-phase import and portfolio routes`
7. `feat(frontend): upload, review, dashboard with Recharts`
8. `docs: README with CAS setup instructions`

---

### ## Key implementation notes

- **Never use float** for money — parse mfapi NAV strings with `Decimal`; casparser already returns `Decimal` in v1.3 models.
- **XIRR cashflow sign convention:** purchases/outflows negative; redemptions + current market value positive (terminal dated `as\_of`).
- **Grandfathering:** for equity lots purchased before `2018-02-01`, compare FIFO cost vs NAV on `2018-01-31` from cached history; use higher as cost basis for LTCG estimate.
- **Re-upload overlapping CAS:** dedupe reports "12 new, 48 duplicates skipped" — no double-counting.
- **NSDL/CDSL demat CAS:** out of v1 scope — return clear message if detected.

```mermaid

sequenceDiagram

participant User

participant UI

participant API

participant Parser

participant Enrich

participant DB

User->>UI: Upload PDF + password

UI->>API: POST /imports/parse

API->>Parser: read_cas_pdf temp file

Parser-->>API: CASData JSON

API->>Enrich: resolve AMFI + NAV

Enrich-->>API: preview with confidence

API-->>UI: review payload
User->>UI: Confirm matches
UI->>API: POST /imports/confirm
API->>DB: insert txns with dedupe
DB-->>API: added/skipped counts
API-->>UI: success
UI->>API: GET portfolio/*
API-->>UI: dashboard data
...